CURSO BACKEND JAVA & QUARKUS — AULA 04

Arquitetura de Banco de Dados: SQL, NoSQL, Conectividade e Integração

Um **Banco de Dados (SGBD)** é um sistema de software encarregado de armazenar, organizar, recuperar e garantir a integridade dos dados de uma aplicação. Nesta aula, entenderemos a fundo como os bancos funcionam desde a infraestrutura de rede até a manipulação de dados em SQL/NoSQL e a integração com Java/Quarkus.

1. SQL VS NOSQL: ENTENDENDO OS PARADIGMAS

A escolha entre um banco relacional (SQL) e não-relacional (NoSQL) depende da estrutura dos dados e dos requisitos de consistência e escalabilidade do projeto.

Característica	Banco Relacional (SQL)	Banco Não-Relacional (NoSQL)
Estrutura de Dados	Tabelas rígidas com linhas e colunas (Schema-first).	Documentos (JSON/BSON), Chave-Valor, Grafos ou Colunas.
Garantia Transacional	**ACID** (Atomicidade, Consistência, Isolamento, Durabilidade).	**BASE** (Basic Availability, Soft-state, Eventual consistency).
Escalabilidade	Predominantemente **Vertical** (aumentar CPU/RAM do servidor).	Predominantemente **Horizontal** (distribuir em nós/clusters).
Relacionamentos	Nativos via Chaves Estrangeiras (`FOREIGN KEY`) e `JOIN`s.	Dados aninhados/embutidos ou referências manuais simples.
Exemplos Populares	PostgreSQL, MySQL, Oracle, SQL Server, H2 Database.	MongoDB, Redis, Cassandra, DynamoDB, Neo4j.

2. COMO UM BANCO DE DADOS SOBE, RODA E FICA EXPOSTO

2.1. O Ciclo de Vida do Processo e Exposição de Porta TCP/IP

Um banco de dados é um **processo servidor** em execução. Para permitir que o Java (ou qualquer cliente) converse com ele, o servidor precisa se vincular a uma interface de rede e escutar em uma porta TCP/IP específica:

- **Processo Daemon:** O motor (ex: `postgres` ou `mongod`) é iniciado no sistema operacional e aloca memória RAM para buffer e cache de disco.
- **Binding de Rede:** O SGBD escuta em um endereço IP. Se configurado para `127.0.0.1` (localhost), aceita apenas conexões locais. Se configurado para `0.0.0.0`, aceita conexões externas de qualquer interface de rede.
- **Portas Padrão:** PostgreSQL (`5432`), MySQL (`3306`), MongoDB (`27017`), Redis (`6379`).

2.2. Subindo um Banco via Docker & String de Conexão

Na arquitetura moderna, subimos bancos de dados isolados em containers Docker. A comunicação entre o Java e o banco utiliza uma **Connection String** (URI de Conexão) contendo protocolo, host, porta, banco, usuário e senha.

```
# Subindo um container PostgreSQL exposto na porta 5432 do host local:
docker run --name pg-quarkus -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=appdb -p 5432:5432 -d
postgres:15

# Formato genérico da String de Conexão JDBC (Java Database Connectivity):
jdbc:postgresql://localhost:5432/appdb?user=postgres&password=secret
```

3. LINGUAGEM SQL APROFUNDADA (RELACIONAL)

A linguagem SQL (Structured Query Language) é dividida em subgrupos: **DDL** (Data Definition Language - estrutura) e **DML** (Data Manipulation Language - dados).

3.1. DDL: Criação de Tabelas, Restrições e Chaves Estrangeiras

O DDL define a estrutura lógica. Chaves primárias garantem unicidade e chaves estrangeiras asseguram a integridade referencial entre tabelas.

```
-- Criação da tabela de Categorias
CREATE TABLE tb_categoria (
    id BIGSERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL UNIQUE
);

-- Criação da tabela de Produtos vinculada à Categoria
CREATE TABLE tb_produto (
    id BIGSERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    preco DECIMAL(10, 2) NOT NULL CHECK (preco > 0),
    categoria_id BIGINT NOT NULL,
    data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_produto_categoria FOREIGN KEY (categoria_id) REFERENCES tb_categoria(id)
);
```

3.2. DML: Inserção, Atualização, Deleção e Consultas Avançadas

Comandos de escrita (`INSERT`, `UPDATE`, `DELETE`) e leitura complexa com cruzamentos (`JOIN`), filtros (`WHERE`) e agrupamentos (`GROUP BY`).

```
-- Inserindo Dados
INSERT INTO tb_categoria (nome) VALUES ('Eletrônicos'), ('Livros');
INSERT INTO tb_produto (nome, preco, categoria_id) VALUES ('Notebook', 4500.00, 1), ('Mouse', 150.00,
1);

-- Atualização e Deleção Seguras (SEMPRE com WHERE!)
UPDATE tb_produto SET preco = 4200.00 WHERE id = 1;
DELETE FROM tb_produto WHERE id = 2;

-- Consulta Avançada com JOIN, Agrupamento e Filtro de Agregado (HAVING)
SELECT
    c.nome AS categoria,
    COUNT(p.id) AS total_produtos,
    AVG(p.preco) AS preco_medio
FROM tb_categoria c
INNER JOIN tb_produto p ON c.id = p.categoria_id
WHERE p.preco > 50.00
GROUP BY c.nome
HAVING COUNT(p.id) >= 1
ORDER BY preco_medio DESC;
```

3.3. Principais Tipos de JOINs no SQL

- **INNER JOIN:** Retorna apenas os registros que possuem correspondência exata em ambas as tabelas.
- **LEFT JOIN:** Retorna todos os registros da tabela da esquerda, mesmo que não haja correspondência na direita (preenche com `NULL`).
- **RIGHT JOIN:** Retorna todos os registros da tabela da direita e os correspondentes da esquerda.

4. UNIVERSO NOSQL: ORIENTADO A DOCUMENTOS E CHAVE-VALOR

Bancos NoSQL abrem mão de schemas fixos para obter alta flexibilidade de escrita e leitura em escala. O modelo mais comum no desenvolvimento backend é o **Document Database (MongoDB)**.

4.1. Operações no MongoDB (Documentos JSON / BSON)

No MongoDB, dados são armazenados como documentos flexíveis dentro de **Collections** (o equivalente às tabelas no SQL).

```
// Inserindo um documento flexível na collection 'produtos'
db.produtos.insertOne({
    nome: "Smartphone Galactic",
    preco: 2999.90,
    tags: ["tech", "mobile"],
    especificacoes: { ram: "8GB", armazenamento: "128GB" },
    ativo: true
});

// Consultas com Operadores de Comparação ($gt: maior que, $in: contido em)
db.produtos.find({ preco: { $gt: 1000.00 }, ativo: true });

// Atualização de Documento usando o operador $set
db.produtos.updateOne(
    { nome: "Smartphone Galactic" },
    { $set: { preco: 2799.90, "especificacoes.ram": "12GB" } }
);

// Agregação (Pipeline similar ao GROUP BY + SUM do SQL)
db.produtos.aggregate([
    { $match: { ativo: true } },
    { $group: { _id: "$tags", totalPreco: { $sum: "$preco" }, qtd: { $sum: 1 } } }
]);
```

5. MODELO CHAVE-VALOR COM REDIS (CACHE EM MEMÓRIA)

5.1. Comandos de Cache e Alta Performance no Redis

O **Redis** armazena dados em memória RAM no formato Chave-Valor, sendo utilizado para cache de sessão, limitação de taxa (Rate Limiting) e dados temporários de altíssima velocidade.

```
# Definindo uma chave de string com tempo de expiração (TTL) de 60 segundos
SET sessao:usuario:1001 "{"nome":"Ana", "perfil":"ADMIN"}" EX 60

# Lendo a chave gravada
GET sessao:usuario:1001

# Operações em Listas / Filas (Push e Pop)
RPUSH fila:emails "welcome@cliente.com"
LPOP fila:emails
```

6. COMO O JAVA / QUARKUS INTERAGE COM O BANCO DE DADOS

A aplicação Java não executa comandos diretamente na rede. Ela utiliza drivers (JDBC para SQL ou Mongo Driver) e abstrações de alto nível como **Panache (Active Record / Repository)**.

6.1. Configuração no Quarkus (`application.properties`) e Dev Services

O Quarkus possui o incrível recurso **Dev Services**: em modo de desenvolvimento (`quarkus dev`), se você não configurar uma URL de banco, ele sobe automaticamente um container Docker do PostgreSQL ou MongoDB via Testcontainers sem nenhuma intervenção manual!

```
# Configuração de Banco Relacional PostgreSQL em Produção
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=postgres
quarkus.datasource.password=secret
quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5432/appdb
quarkus.datasource.jdbc.max-size=16  # Pool de Conexões HikariCP

# Geração Automática do Schema de Banco (drop-and-create, update, validate)
quarkus.hibernate-orm.database.generation=update
```

6.2. Interação Java + SQL usando Hibernate ORM Panache

O desenvolvedor manipula objetos Java, e o Panache traduz os métodos em comandos SQL `INSERT`, `SELECT`, `UPDATE` e `DELETE` transacionais:

```
package com.empresa.service;

import com.empresa.domain.Produto;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.transaction.Transactional;
import java.util.List;

@ApplicationScoped
public class ProdutoRepositoryService {

    @Transactional // Inicia a transação JDBC. Ao final, executa COMMIT ou ROLLBACK em falhas
    public void processarCatalogo() {
        // Exemplo de INSERT via Objeto
        Produto p = new Produto();
        p.nome = "Teclado Mecânico";
        p.preco = new BigDecimal("350.00");
        p.persist(); // Gera: INSERT INTO tb_produto ...

        // Exemplo de SELECT com filtros HQL/Panache
        List<Produto> baratos = Produto.list("preco < ?1", new BigDecimal("500.00"));

        // Exemplo de UPDATE em lote
        Produto.update("preco = preco * 1.10 WHERE status = 'ATIVO'");
    }
}
```

6.3. Interação Java + NoSQL usando Panache MongoDB

Para NoSQL, basta alterar a extensão para `quarkus-mongodb-panache` e estender `PanacheMongoEntity`. A sintaxe de métodos Java permanece quase idêntica!